ECSE 318 Lab 4, Problem 2: Shift-Add Multiplier

Group 9. Structural 4-bit VHDL model, compiled and simulated in ModelSim Intel FPGA Starter Edition 2020.1.

Compilation: vcom -2008 adder.vhdl dff.vhdl MultiplierVHDL.vhdl MultiplierVHDL_tb.vhdl

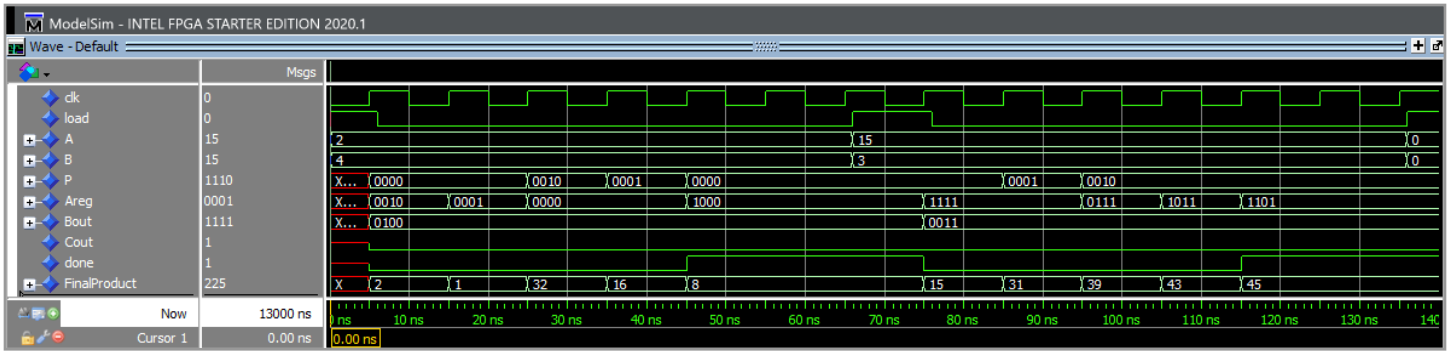
```
Model Technology ModelSim - Intel FPGA Edition vcom 2020.1 Compiler 2020.02 Feb 28 2020
Start time: 17:34:55 on Nov 01,2026
-- Compiling entity adder
-- Compiling architecture Behavioral of adder
-- Compiling entity dff
-- Compiling architecture Behavioral of dff
-- Compiling entity MultiplierVHDL
-- Compiling architecture Behavioral of MultiplierVHDL
-- Loading entity adder
-- Loading entity dff
-- Compiling entity MultiplierVHDL_tb
-- Compiling architecture sim of MultiplierVHDL_tb
-- Loading entity MultiplierVHDL
End time: 17:34:55 on Nov 01,2026, Elapsed time: 0:00:00
Errors: 0, Warnings: 0
```

Simulation transcript

```
# vsim -c -do "run -all; quit -f" MultiplierVHDL_tb
# Start time: 17:34:57 on Nov 01,2026
# Loading work.multiplierVHDL_tb(sim)
# Loading work.multiplierVHDL(behavioral)
# Loading work.adder(behavioral)
# Loading work.dff(behavioral)
# run -all
# ** Note: 2 * 4 = 8
#    Time: 46 ns  Iteration: 0  Instance: /multiplierVHDL_tb
# ** Note: 15 * 3 = 45
#    Time: 116 ns  Iteration: 0  Instance: /multiplierVHDL_tb
# ** Note: failures: 0
#    Time: 12936 ns  Iteration: 0  Instance: /multiplierVHDL_tb
# End time: 17:34:57 on Nov 01,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
```

The testbench runs the two required cases, then all 256 4-bit input pairs. "failures: 0" means every product was correct and took exactly 4 clocks.

Waveform, 0 to 140 ns (empty rows and the side panel are cropped out of the ModelSim window)



Case	FinalProduct after each clock	Result
2 * 4	2 (loaded), 1, 32, 16, 8	8, done goes high at 45 ns
15 * 3	15 (loaded), 31, 39, 43, 45	45, done goes high at 115 ns

Problem 3
Since the circuit is combinational, the time it takes to complete one operation is determined by the longest path through the circuit,
which we can trace through the number of gates it needs to go through.

Per the problem, we assume each gate has one unit of delay.

We can look at this cell by cell by looking at how many gates there are in one CAS:

In fa.vhd, we have the following:

cout <= (a and b) or (a and cin) or (b and cin);
sum <= a xor b xor cin;

Both cout and sum go through two gates. Therefore, we have two units of delay per fa. We have one fa in each cas cell. So each cas cell goes through 2 units of delay.

We have four of these cells per row, so that ends up being 2 * 4 units of delay from the cas cells in one row. Additionally, in each cell, we have a xor gate (b_control <= m xor b;) which runs in parallel per row. Therefore, we add a +1 unit delay per row, to give us 8 + 1 = 9 units of delay per row. Since we have 5 of these rows, we have 9 * 5 = 45 units of delay.

Problem 4

Adding a sequence of 10 8-bit binary numbers using a CSA design uses an 8 stage CSA followed by a regular adder if all the 3 inputs adders computed their outputs sequentially. However, if all the 3 input adders computes their CSA outputs in parallel, then a 5 stage CSA followed by a regular adder can be used to add the sequence of numbers.

This is because the inital 10 numbers can be group into sets of 3s with the remainder number temporarily unattended, then each sets of 3 can have their inputs summed in a CSA stage resulting in 7 remaining operands (3 sums, 3 carry outs and the remaining 10th number).

If this process is recursively performed for the 7 operands and their CSA stage outputs then then number of operands after all the parallel CSA stage changes from 10 to 7 to 5 to 4 to 3 to 2 showing the 5 stages.